

AST Code Completion Model

Общее описание проекта

Проект представляет собой интеллектуальную систему автоматического продолжения Python-кода на основе Transformer Language Model, обученной на структурированном AST-представлении программ.

Главной особенностью системы является использование не обычного текстового представления исходного кода, а Abstract Syntax Tree (AST), позволяющего модели учитывать синтаксическую структуру программы, взаимосвязи между конструкциями языка и вложенность элементов кода.

В классических language models код рассматривается как последовательность символов или токенов. В данной системе применяется структурный подход, при котором исходный Python-код сначала преобразуется в синтаксическое дерево, после чего линейризуется в специальную последовательность AST-токенов. Это позволяет модели лучше понимать:

- структуру функций;
- вложенность операторов;
- связи между выражениями;
- синтаксические зависимости;
- контекст выполнения кода.

Система предназначена для решения задач:

- code completion;
- генерации продолжения функций;
- восстановления незавершённого кода;
- автоматического дополнения AST;
- интеллектуального автодополнения в IDE;
- генерации синтаксически корректного Python-кода.

Цель проекта

Целью проекта является разработка модели генерации программного кода, использующей структурное AST-представление вместо традиционного token-level подхода.

Основные задачи проекта:

- исследование эффективности AST-based language models;
- повышение синтаксической корректности генерации;
- уменьшение количества некорректного кода;
- обучение модели структурному представлению программ;
- реализация устойчивой autoregressive генерации.

Архитектура системы

Система включает несколько основных компонентов:

1. Parser Pipeline

Модуль обработки исходного кода выполняет:

1. токенизацию Python-кода;
2. синтаксический разбор;
3. построение AST;
4. преобразование дерева в JSON-структуру;
5. линейризацию AST.

2. AST Linearizer

Для преобразования дерева в последовательность используется DFS traversal.

Каждый узел AST кодируется специальными структурными токенами:

```
ENTER_NodeType
EXIT_NodeType
VALUE_*
ID_*
ARITY_n
```

Такой формат позволяет сохранять информацию о:

- типе узла;
- структуре дерева;
- значениях идентификаторов;
- вложенности конструкций.

3. Transformer Decoder

Основой модели является autoregressive Transformer decoder.

Архитектура включает:

- Multi-Head Self Attention;
- positional embeddings;
- residual connections;
- LayerNorm;
- Feed Forward блоки;
- causal masking.

Модель выполняет предсказание следующего AST-токена на основе предыдущего контекста.

Пайплайн обработки данных

Полный процесс обработки программы:

```
Python source code
→ tokenize
→ ast.parse()
→ JSON AST tree
→ AST linearization
→ vocabulary encoding
→ Transformer input
→ autoregressive generation
→ AST reconstruction
→ Python code reconstruction
```

Используемые методы

AST-based Representation

Вместо классического token-level представления используется структурное AST-кодирование.

Преимущества подхода:

- сохранение синтаксической структуры;
 - уменьшение неоднозначности;
 - повышение устойчивости генерации;
 - лучшая обработка вложенных конструкций.
-

Autoregressive Language Modeling

Модель обучается предсказывать следующий AST-токен:

$$P(t_i \mid t_1, t_2, \dots, t_{i-1})$$

Используется causal attention mask, исключающая доступ к будущим токенам.

Cross Entropy Optimization

В качестве функции потерь используется:

CrossEntropyLoss

с masking PAD-токенов.

Gradient Accumulation

Для эффективного обучения при ограниченной памяти GPU применяется gradient accumulation.

Это позволяет эмулировать больший effective batch size.

Learning Rate Warmup

Используется постепенное увеличение learning rate на начальных этапах обучения для стабилизации оптимизации.

Датасеты

Training Dataset

python100k_train.json

Содержит AST-представления Python-программ.

Validation Dataset

python50k_eval.json

Используется для:

- validation loss;
 - perplexity evaluation;
 - early stopping.
-

Архитектурные параметры модели

Параметр	Значение
d_model	384
n_heads	6
n_layers	6
ff_mult	4
dropout	0.05
seq_len	1024
vocab_size	20000

Параметры обучения

Параметр	Значение
epochs	12
batch_size	4
accum_steps	16
learning_rate	2e-4
min_lr	2e-5
warmup_steps	2500
weight_decay	0.05
grad_clip	0.5
optimizer	AdamW

Параметры генерации

Параметр	Значение
temperature	0.6
top_k	24
top_p	0.9
repetition_penalty	1.15
max_new_tokens	96

Используемые специальные токены

Токен	Назначение
<PAD>	padding
<UNK>	unknown token
<BOS>	beginning of sequence
<EOS>	end of sequence
<PROMPT>	prompt boundary
<COMPLETION>	completion boundary

Алгоритм обучения

Процесс обучения включает следующие этапы:

- 1. Загрузка JSONL датасета.
- 2. Построение byte offsets.
- 3. Потокочное чтение данных.
- 4. Парсинг AST.
- 5. Линейризация дерева.
- 6. Кодирование токенов.
- 7. Формирование batch.
- 8. Forward pass модели.
- 9. Вычисление CrossEntropyLoss.
- 10. Backpropagation.
- 11. Gradient clipping.
- 12. Optimizer step.
- 13. Scheduler update.
- 14. Validation evaluation.

Метрики качества

Для оценки качества модели используются:

Training Loss

Средняя ошибка предсказания следующего токена на обучающем наборе.

Validation Loss

Ошибка на validation dataset.

Perplexity (PPL)

Метрика качества language model:

$$PPL = \exp(loss)$$

Меньшее значение PPL соответствует более качественной генерации.

График динамики обучения

Ниже приведён график изменения training loss по эпохам.

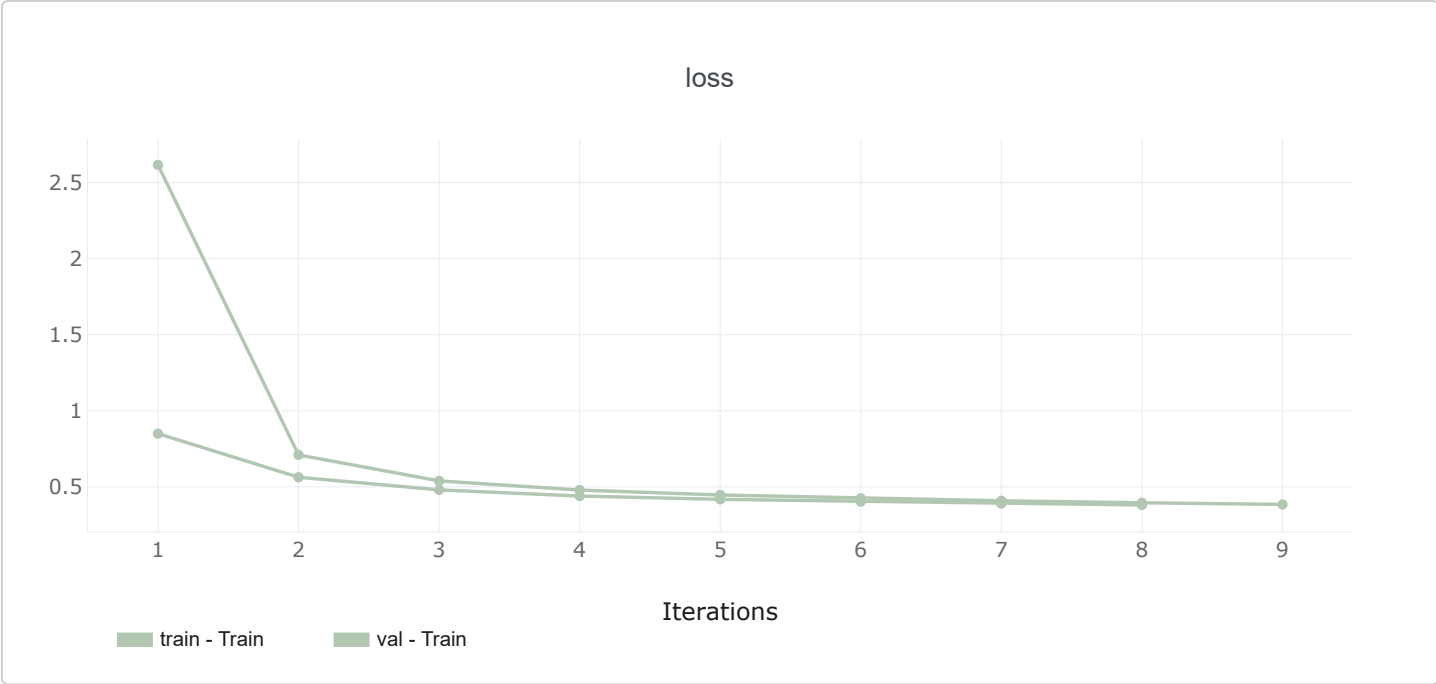


График изменения Perplexity

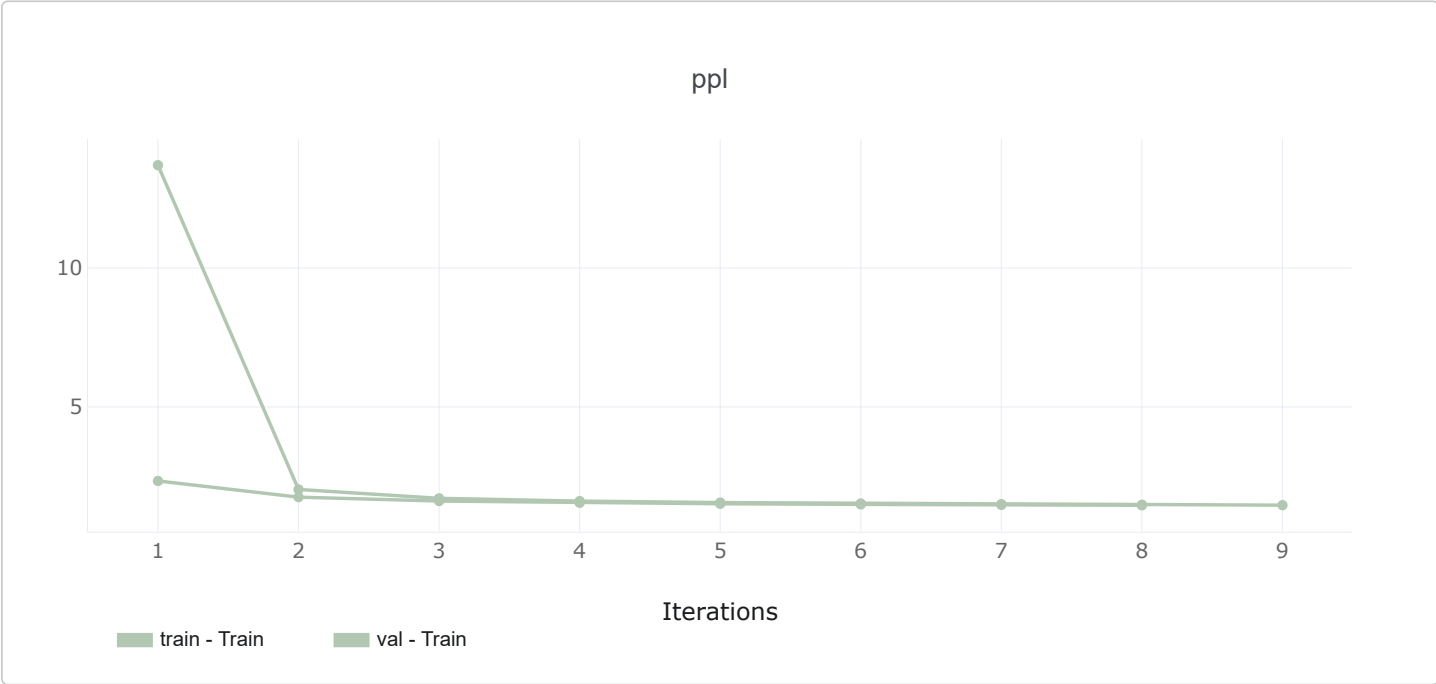
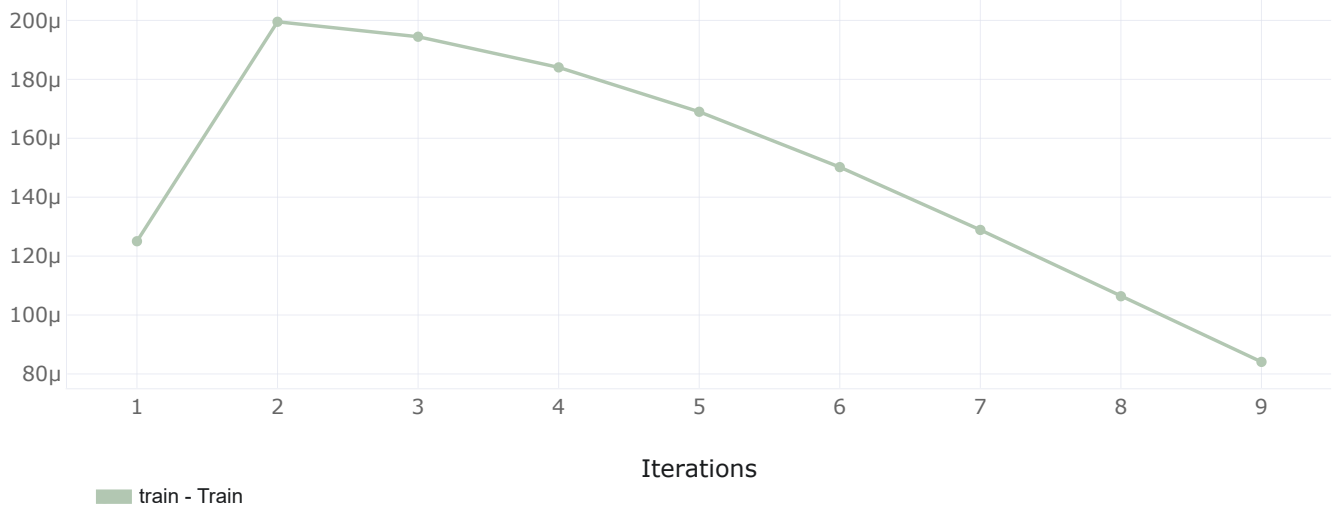


График Learning Rate

lr



Особенности реализации

Streaming Dataset

Используется потоковое чтение данных без полной загрузки датасета в оперативную память.

AST Reconstruction

Система поддерживает:

- восстановление AST;
- обратную генерацию Python-кода;
- реконструкцию синтаксической структуры.

Fault Tolerance

Реализована обработка:

- invalid JSON;
- parsing errors;
- tokenizer failures;
- corrupted samples.

Inference Pipeline

Во время генерации выполняются:

1. Parsing input code.
2. AST conversion.
3. Token encoding.
4. Transformer inference.
5. Sampling.
6. AST reconstruction.
7. Python code generation.

Web Interface

Проект содержит Flask-based web interface.

Функциональность:

- ввод пользовательского кода;
- генерация продолжения;
- выбор параметров inference;
- отображение результата.

Используемые технологии

Категория	Технология
Programming Language	Python
Deep Learning Framework	PyTorch
Web Framework	Flask
Experiment Tracking	ClearML
Parsing	AST
Tokenization	tokenize
Dataset Format	JSONL

Практическое применение

Система может использоваться:

- в интеллектуальных IDE;
- системах автодополнения;
- AI code assistants;
- образовательных платформах;
- системах анализа программного кода;
- инструментах автоматической генерации кода.

Заключение

В результате работы была разработана AST-based language model для интеллектуального продолжения Python-кода.

Использование структурного представления Abstract Syntax Tree позволило повысить синтаксическую устойчивость генерации и обеспечить более глубокое понимание структуры программы по сравнению с классическими token-level моделями.

Реализованная система поддерживает полный цикл:

- parsing;
- AST linearization;
- Transformer training;
- autoregressive inference;
- reconstruction Python code.